

Documento de Desenvolvimento de Software

MomoRogue Engine

Engine especializada para jogos roguelike em C#, SadConsole, scripting e
exportação para executável/instalador

Versão: 0.1 - Documento inicial de arquitetura e requisitos

Data: 11 de junho de 2026

Plataforma alvo inicial: Windows x64

Tecnologias base: C#, .NET, SadConsole, Avalonia, Roslyn, WiX

Status: Planejamento técnico para MVP

Sumário

1	Introdução	3
1.1	Objetivo geral	3
1.2	Objetivos específicos	3
2	Escopo do Produto	3
2.1	Produto proposto	3
2.2	Fora do escopo inicial	4
3	Justificativa Técnica da Stack	4
4	Visão Geral da Arquitetura	5
4.1	Princípio arquitetural principal	5
4.2	Organização da solução	5
5	Módulos da Engine	6
5.1	MomoRogue.Engine	6
5.2	MomoRogue.SadConsole	6
5.3	MomoRogue.Runtime	7
5.4	MomoRogue.Editor	7
5.5	MomoRogue.BuildTool	7
6	Modelo de Projeto do Jogo	7
6.1	Arquivo game.mrgproj	7
7	Requisitos Funcionais	8
8	Requisitos Não Funcionais	9
9	Casos de Uso	10
9.1	UC-01 - Criar projeto	10
9.2	UC-02 - Programar comportamento em C#	11
9.3	UC-03 - Criar visual script	11
9.4	UC-04 - Exportar jogo	11
10	Pipeline de Build e Exportação	12
10.1	Estratégia de build	12
10.2	Comando base de publicação	13
10.3	Tipos de entrega	13
11	Runtime e Fluxo de Turno	13
11.1	Estados do runtime	14
12	Modelo de Classes Inicial	14
12.1	Exemplo de contratos em C#	14
13	Scripting em C#	15
13.1	Estratégia inicial	15
13.2	Regras para scripts	16

14 Visual Scripting	16
14.1 Modelo do grafo	16
14.2 Código gerado esperado	17
15 Dados do Jogo	17
15.1 Definição de entidade	17
15.2 Save game	17
16 Regras de Negócio	18
17 Critérios de Aceitação do MVP	18
18 Plano de Implementação	19
18.1 Fase 1 - Núcleo jogável	19
18.2 Fase 2 - Roguelike mínimo	19
18.3 Fase 3 - Projeto e dados	19
18.4 Fase 4 - Scripting	19
18.5 Fase 5 - Build tool	20
18.6 Fase 6 - Editor	20
18.7 Fase 7 - Visual scripting	20
19 Riscos e Mitigações	20
20 Estratégia de Testes	21
20.1 Testes unitários	21
20.2 Testes de integração	21
20.3 Testes manuais	21
21 Definição de Pronto	21
22 Roadmap	22
23 Referências Técnicas	22
A Artefatos de Diagrama	23
B Backlog Técnico Inicial	23

1 Introdução

Este documento especifica o desenvolvimento da **MomoRogue Engine**, uma engine própria, especializada em jogos *roguelike* 2D baseados em grade. O objetivo não é criar uma engine genérica no estilo Unity, Godot ou Unreal, mas uma plataforma focada em um domínio menor e mais controlável: mapas tile-based, entidades, turnos, geração procedural, campo de visão, pathfinding, scripts de gameplay e exportação do jogo final para executável ou instalador.

A decisão técnica central é usar **C#** como linguagem principal e **SadConsole** como camada de renderização/input. A lógica da engine deve permanecer desacoplada do SadConsole, permitindo que a engine seja testada sem interface gráfica e, futuramente, possa receber outro renderizador. O SadConsole entra como adaptador visual para desenhar tiles, glyphs, spritesheets e interfaces textuais/semigráficas.

1.1 Objetivo geral

Desenvolver uma engine modular para criação de jogos roguelike, com suporte a edição de projeto, programação em C#, visual scripting, execução em runtime e exportação automatizada para formatos distribuíveis.

1.2 Objetivos específicos

- a) Definir uma arquitetura modular e testável para o núcleo da engine.
- b) Implementar um runtime capaz de carregar projetos de jogo, assets, dados e scripts.
- c) Integrar SadConsole como renderizador tile-based e camada de input.
- d) Permitir scripts em C# compilados junto ao projeto do jogo.
- e) Permitir visual scripting por grafo, inicialmente convertido para C# gerado.
- f) Criar um editor desktop para gerenciar projetos, dados, assets e exportação.
- g) Criar um build tool capaz de gerar executável, pasta portable e instalador.
- h) Manter a engine especializada em roguelikes para reduzir escopo e aumentar viabilidade.

2 Escopo do Produto

2.1 Produto proposto

A MomoRogue Engine será composta por quatro partes principais:

- **Engine:** biblioteca com regras, entidades, turnos, mapas, geração procedural, FOV, pathfinding, save/load e sistemas de jogo.
- **Runtime:** aplicação base que executa jogos criados com a engine.
- **Editor:** ferramenta desktop para criação e configuração de projetos.

- **Build Tool:** ferramenta de empacotamento e exportação para executável, ZIP portable e instalador.

2.2 Fora do escopo inicial

Para manter o projeto viável, ficam fora do MVP:

- Física 2D/3D genérica.
- Animação pixel-perfect complexa fora do grid.
- Editor visual completo equivalente a Unity/Godot.
- Multiplayer online.
- Marketplace de assets ou mods.
- Suporte mobile no primeiro ciclo.
- Scripting não confiável com sandbox robusta.

3 Justificativa Técnica da Stack

A stack proposta privilegia produtividade, integração com tooling moderno e aderência ao tipo de jogo. SadConsole é adequado porque roguelikes geralmente operam em grade, usando caracteres, glyphs ou tiles. C# e .NET facilitam organização modular, reflexão, geração de código, serialização, integração com Roslyn e publicação multiplataforma.

Camada	Escolha	Motivo
Linguagem	C#	Boa ergonomia, tipagem forte, tooling maduro, integração com .NET SDK e Roslyn.
Runtime/base	.NET	Permite publicar aplicações self-contained e single-file com dotnet publish .
Renderização	SadConsole	Adequado a jogos ASCII, ANSI, tile-based e roguelike.
Host gráfico	MonoGame via SadConsole	Permite janela, input e renderização 2D por backend já suportado.
Editor	Avalonia UI	Framework desktop .NET multiplataforma com XAML, MVVM e boa separação de interface/lógica.
Scripting	Roslyn / Micro-soft.CodeAnalysis	Permite compilação/análise de scripts C# e geração de assemblies.
Instalador	WiX Toolset	Geração de MSI/instaladores Windows com controle fino do pacote.
Dados	JSON	Simples, legível, versionável e suficiente para entidades, itens, configs e grafos.

4 Visão Geral da Arquitetura

A arquitetura deve evitar dependência direta entre gameplay e renderizador. O núcleo da engine não deve conhecer SadConsole. Ele apenas expõe estado, eventos e comandos. O adaptador SadConsole lê esse estado e desenha o mundo.

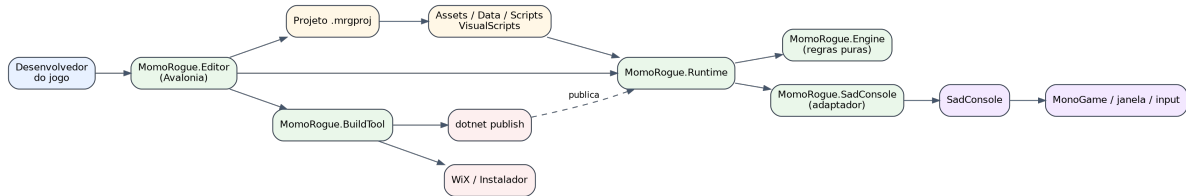


Figura 1: Contexto geral da arquitetura proposta.

4.1 Princípio arquitetural principal

SadConsole é tela; MomoRogue.Engine é regra de jogo.

Essa separação reduz acoplamento e facilita testes automatizados. A engine deve conseguir processar turnos, combate, movimentação, geração procedural e saves sem abrir uma janela gráfica.

4.2 Organização da solução

```

MomoRogue/
src/
  MomoRogue.Engine/
  MomoRogue.SadConsole/
  MomoRogue.Runtime/
  MomoRogue.Editor/
  MomoRogue.BuildTool/
templates/
  BasicRoguelikeProject/
samples/
  DungeonSample/
  SciFiSample/
tests/
  MomoRogue.Engine.Tests/
  MomoRogue.BuildTool.Tests/
docs/
  diagrams/
  
```

Listing 1: Estrutura inicial da solução C#.

5 Módulos da Engine

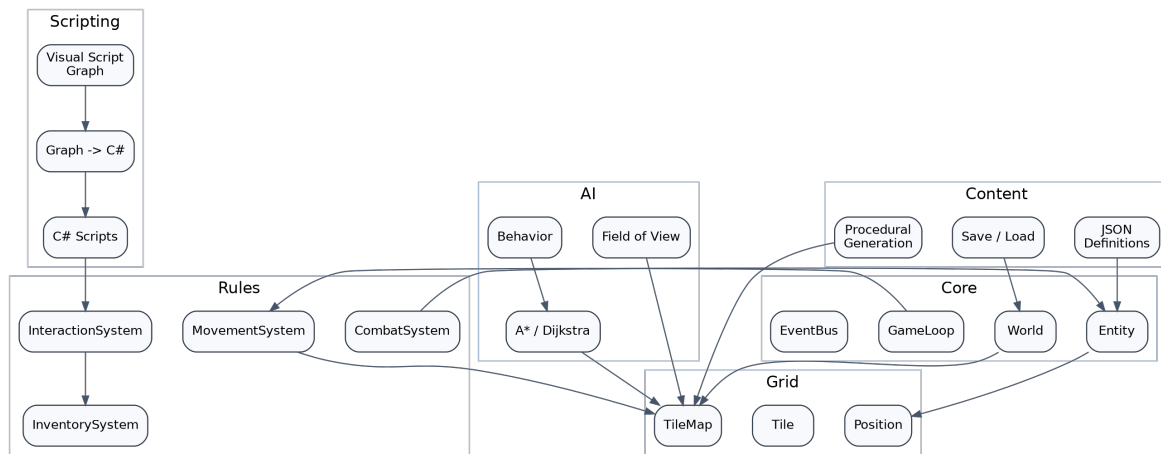


Figura 2: Módulos internos da engine.

5.1 MomoRogue.Engine

Biblioteca pura com as regras e estruturas principais. Não deve depender de SadConsole, Avalonia ou UI.

- **Core:** loop abstrato, mundo, entidades, componentes e eventos.
- **Grid:** tilemap, posição, direção, tamanho, colisão e visibilidade.
- **TurnBased:** gerência de turnos, energia, velocidade e ações.
- **Rules:** movimento, combate, inventário, interação e uso de itens.
- **AI:** comportamentos, pathfinding, Dijkstra map e FOV.
- **ProcGen:** geração procedural de salas, corredores, cavernas e andares.
- **Scripting:** contrato para scripts C# e visual scripts.
- **Persistence:** save/load e serialização do estado de jogo.

5.2 MomoRogue.SadConsole

Adaptador responsável por traduzir estado da engine para tela.

- Renderização de tilemap.
- Renderização de entidades por glyph/tile.
- Camadas de UI: log, inventário, status, menu.
- Tradução de input para comandos da engine.
- Carregamento de spritesheets/tilesets compatíveis com SadConsole.

5.3 MomoRogue.Runtime

Aplicação que executa o jogo final. Ela carrega o pacote do jogo, inicializa a engine, conecta o renderer e inicia o loop.

5.4 MomoRogue.Editor

Aplicação desktop para criar e configurar projetos. O editor não deve ser obrigatório para executar o jogo. Ele é uma ferramenta de produção.

5.5 MomoRogue.BuildTool

Ferramenta de linha de comando ou serviço interno do editor responsável por validar projeto, gerar código, compilar scripts, copiar assets e executar publicação.

6 Modelo de Projeto do Jogo

Um jogo criado na engine deve ser uma pasta versionável, contendo dados, scripts, assets e configuração.

```
MeuJogo/  
  game.mrgproj  
  Assets/  
    Tilesets/  
      dungeon_16x16.png  
    Audio/  
    Fonts/  
  Data/  
    actors.json  
    items.json  
    maps.json  
    factions.json  
  Scripts/  
    PlayerController.cs  
    GoblinAI.cs  
    FireballSpell.cs  
  VisualScripts/  
    door_interaction.graph.json  
    boss_intro.graph.json  
  Build/
```

Listing 2: Estrutura de um projeto de jogo.

6.1 Arquivo game.mrgproj

```
{  
  "name": "Meu Roguelike",  
  "id": "meu-roguelike",  
  "version": "0.1.0",  
  "engineVersion": "0.1.0",  
}
```



```
"entryScene": "dungeon_start",
"tileSize": 16,
"target": "win-x64",
"outputType": "portable",
"scripting": {
  "compileCSharpScripts": true,
  "compileVisualScripts": true
}
}
```

Listing 3: Exemplo de configuração do projeto.

7 Requisitos Funcionais

ID	Nome	Descrição
RF-01	Criar projeto	O editor deve permitir criar um novo projeto roguelike a partir de um template.
RF-02	Abrir projeto	O editor e o runtime devem carregar um projeto existente por meio do arquivo <code>game.mrgproj</code> .
RF-03	Renderizar mapa	O runtime deve renderizar um mapa baseado em grade usando SadConsole.
RF-04	Suportar tiles/glyphs	A engine deve permitir representar entidades e tiles por glyphs, caracteres ou sprites de tileset.
RF-05	Gerenciar entidades	A engine deve criar, consultar, atualizar e remover entidades do mundo.
RF-06	Componentes	Entidades devem poder receber componentes como posição, renderização, vida, inventário, AI e colisão.
RF-07	Movimento e colisão	A engine deve validar movimento com base em limites do mapa, paredes, entidades bloqueantes e regras de interação.
RF-08	Sistema de turnos	O runtime deve executar turnos do jogador e de entidades controladas por IA.
RF-09	Combate básico	A engine deve fornecer regras mínimas de ataque, dano, vida e morte.
RF-10	Geração procedural	A engine deve gerar dungeons simples com salas e corredores.
RF-11	Campo de visão	A engine deve calcular tiles visíveis, explorados e desconhecidos.
RF-12	Pathfinding	A engine deve oferecer caminho para IA por A* ou Dijkstra map.
RF-13	Scripts C#	O projeto deve aceitar scripts C# compilados junto ao jogo.
RF-14	Visual scripting	O editor deve permitir criar grafos visuais que geram código C#.

RF-15	Playtest	O editor deve permitir executar o jogo em modo de teste.
RF-16	Exportar executável	O build tool deve gerar executável Windows x64 a partir do projeto.
RF-17	Exportar ZIP portable	O build tool deve gerar uma pasta ou arquivo ZIP executável sem instalação.
RF-18	Gerar instalador	O build tool deve gerar instalador MSI/EXE em ciclo posterior ao MVP inicial.
RF-19	Save/load	O runtime deve salvar e carregar estado do jogo.
RF-20	Logs de erro	Editor e build tool devem exibir erros de validação, script e exportação.

8 Requisitos Não Funcionais

ID	Categoria	Descrição
RNF-01	Modularidade	O núcleo da engine deve ser independente de SadConsole, Avalonia e BuildTool.
RNF-02	Testabilidade	Regras de jogo devem ser testáveis por testes unitários sem renderização gráfica.
RNF-03	Desempenho	O MVP deve manter 60 FPS na renderização de mapas pequenos/médios, mesmo com turnos processados sob demanda.
RNF-04	Portabilidade	A arquitetura deve permitir exportação futura para Linux e macOS, mesmo que o MVP foque Windows x64.
RNF-05	Manutenibilidade	Cada módulo deve ter responsabilidade clara e baixo acoplamento.
RNF-06	Reprodutibilidade	Geração procedural deve aceitar seed para reproduzir mapas.
RNF-07	Segurança	Scripts C# devem ser tratados como código confiável no MVP; mods não confiáveis exigem política futura de sandbox.
RNF-08	Observabilidade	Build e runtime devem gerar logs legíveis de erro, warning e eventos importantes.
RNF-09	Versionamento	Projetos devem declarar versão da engine e permitir migração futura de schema.
RNF-10	Usabilidade	O editor deve priorizar fluxos simples: criar projeto, editar dados, testar e exportar.

9 Casos de Uso

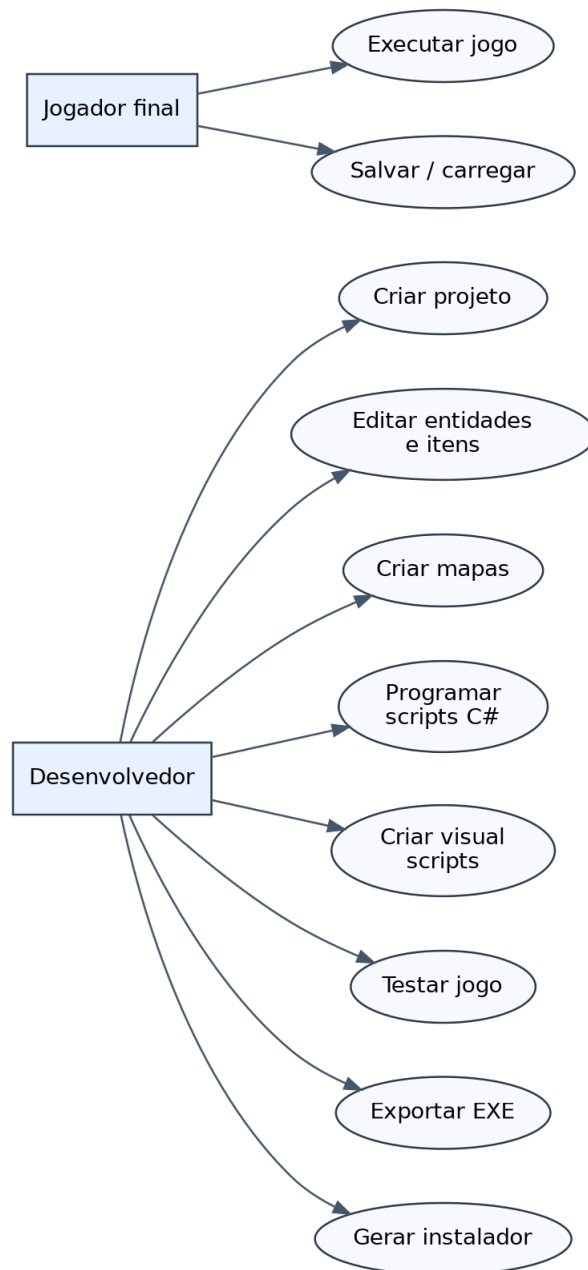


Figura 3: Casos de uso principais.

9.1 UC-01 - Criar projeto

Ator principal: Desenvolvedor.

Fluxo: abre o editor, escolhe template, define nome, pasta e configurações iniciais.

Resultado: pasta de projeto criada com estrutura inicial, arquivos JSON, scripts exemplo e assets mínimos.

9.2 UC-02 - Programar comportamento em C#

Ator principal: Desenvolvedor.

Fluxo: cria um arquivo `.cs` na pasta `Scripts/`, implementa uma classe derivada de contrato da engine, compila pelo editor/build tool.

Resultado: o comportamento fica disponível para entidades do jogo.

9.3 UC-03 - Criar visual script

Ator principal: Desenvolvedor.

Fluxo: abre editor de grafos, conecta eventos, condições e ações.

Resultado: o grafo é salvo como JSON e convertido para C# gerado no build.

9.4 UC-04 - Exportar jogo

Ator principal: Desenvolvedor.

Fluxo: seleciona perfil de build, valida projeto, gera código, compila scripts, publica runtime e empacota.

Resultado: executável, pasta portable ou instalador.

10 Pipeline de Build e Exportação

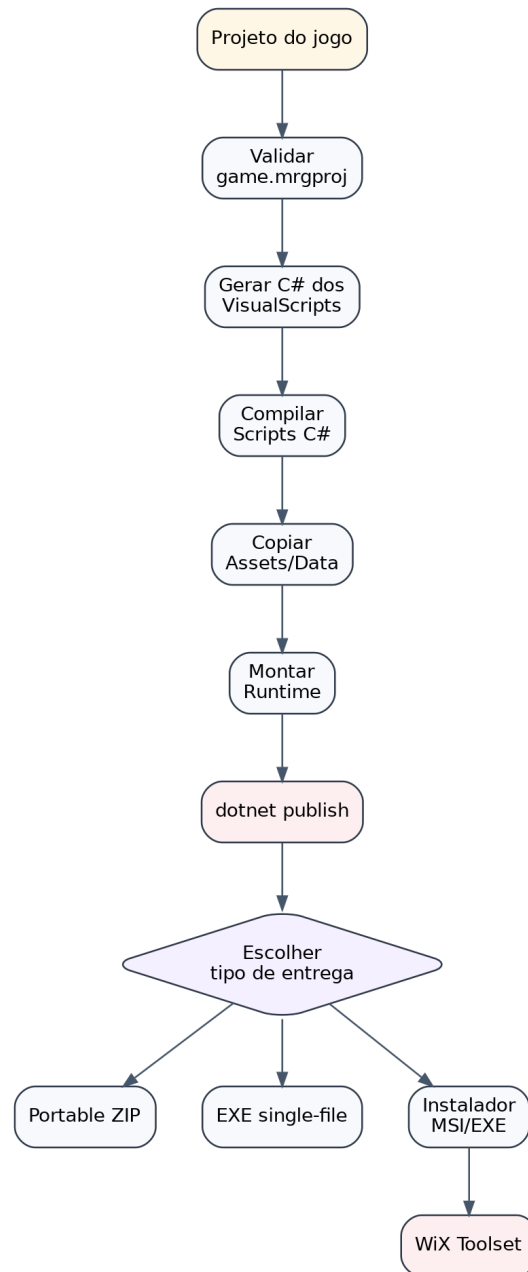


Figura 4: Pipeline de exportação do projeto para executável ou instalador.

10.1 Estratégia de build

A engine não precisa implementar um compilador próprio. Ela deve orquestrar o tooling do .NET:

1. Validar o arquivo `game.mrgproj`.
2. Validar JSONs de dados.
3. Gerar C# a partir de visual scripts.

4. Criar ou atualizar o projeto `.csproj` temporário do jogo.
5. Compilar scripts C# e código gerado.
6. Copiar assets e dados para a saída.
7. Executar `dotnet publish` com perfil definido.
8. Opcionalmente executar empacotamento MSI/EXE por WiX.

10.2 Comando base de publicação

```
dotnet publish MeuJogo.Runtime.csproj \
  -c Release \
  -r win-x64 \
  --self-contained true \
  /p:PublishSingleFile=true \
  /p:PublishTrimmed=false
```

Listing 4: Comando conceitual de exportação Windows x64.

10.3 Tipos de entrega

Tipo	Quando usar	Observações
Pasta portable	MVP e testes	Mais simples; facilita debugar assets e DLLs.
ZIP portable	Distribuição simples	Compacta a pasta publicada.
EXE single-file	Distribuição limpa	Pode exigir cuidado com assets externos e extração.
Instalador MSI/EXE	Distribuição final	Deve ser feito após estabilidade do runtime e do formato do projeto.

11 Runtime e Fluxo de Turno

O runtime executa um ciclo orientado a ações. O jogo não precisa atualizar lógica pesada a cada frame; a lógica de roguelike geralmente avança quando o jogador executa uma ação.

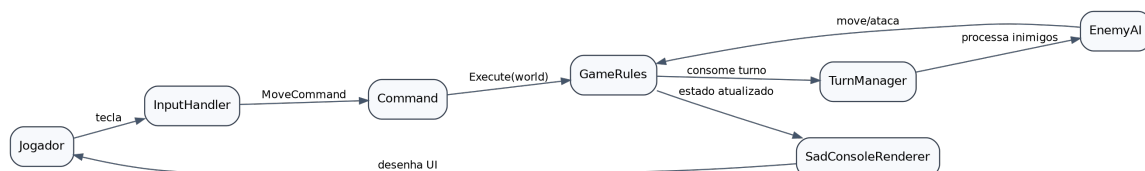


Figura 5: Sequência de processamento de um turno.

11.1 Estados do runtime

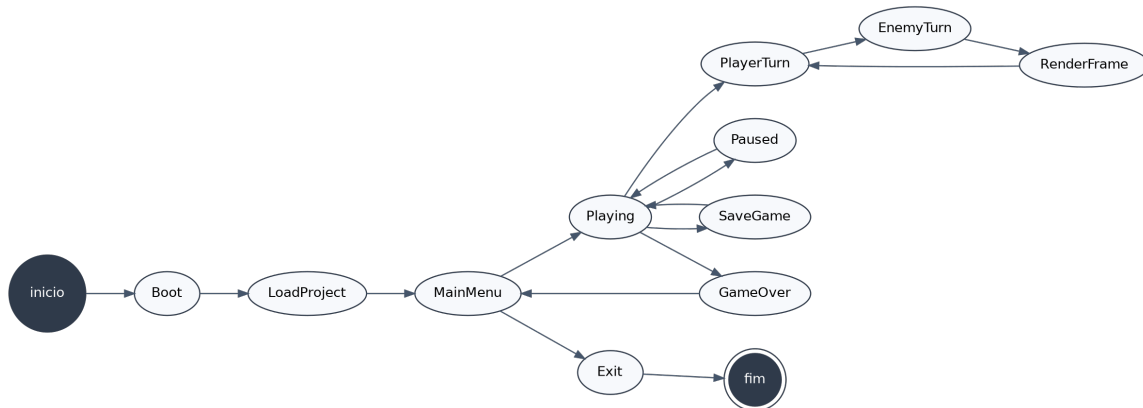


Figura 6: Estados principais do runtime.

12 Modelo de Classes Inicial

A engine deve começar simples, com um modelo de entidades e componentes pragmático. Não é necessário implementar ECS puro no MVP. O ideal é usar entidades com componentes tipados e sistemas bem separados.

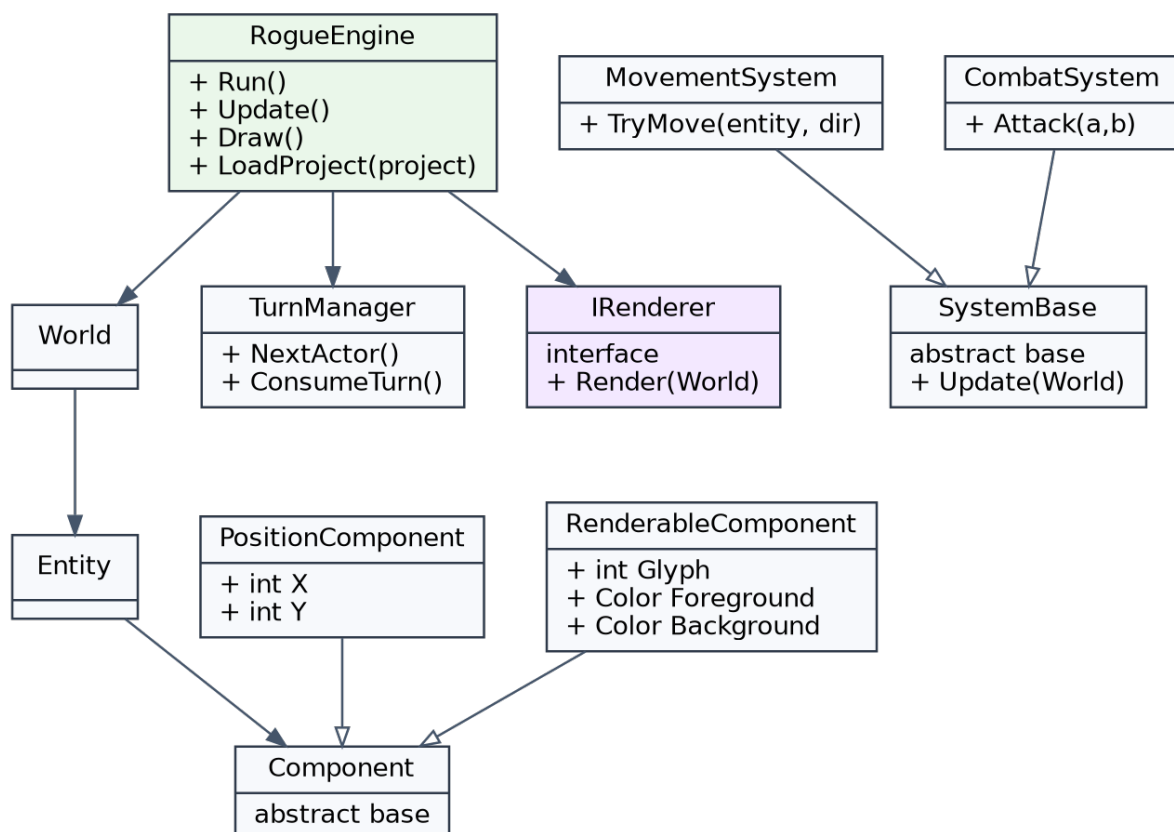


Figura 7: Modelo conceitual de classes do núcleo da engine.

12.1 Exemplo de contratos em C#

```
public interface IRenderer
{
    void Render(World world);
}

public sealed class RogueEngine
{
    private readonly World world;
    private readonly IRenderer renderer;
    private readonly TurnManager turnManager;

    public void Update(ICommand? command)
    {
        if (command is null)
            return;

        command.Execute(world);
        turnManager.ProcessNonPlayerActors(world);
    }

    public void Draw()
    {
        renderer.Render(world);
    }
}
```

Listing 5: Exemplo conceitual de contrato de renderer.

13 Scripting em C#

13.1 Estratégia inicial

O MVP deve priorizar scripts C# compilados junto com o jogo, não execução dinâmica irrestrita em runtime. Isso é mais simples, performático e previsível.

```
using MomoRogue.Engine;

public sealed class GoblinAI : ActorBehavior
{
    public override void OnTurn(Actor self, World world)
    {
        Entity player = world.Player;

        if (self.Position.DistanceTo(player.Position) <= 1)
        {
            self.Attack(player);
            return;
        }

        self.MoveToward(player.Position);
    }
}
```



```
}

```

Listing 6: Exemplo de comportamento de inimigo em C#.

13.2 Regras para scripts

- Scripts devem implementar interfaces ou herdar classes abstratas da engine.
- Scripts não devem acessar diretamente SadConsole.
- O build tool deve compilar scripts e exibir erros com arquivo, linha e coluna.
- Scripts do MVP são considerados código confiável do próprio desenvolvedor.
- Suporte a mods externos deve ser tratado em fase posterior.

14 Visual Scripting

O visual scripting deve ser implementado como um grafo que gera C#. Essa decisão evita criar um interpretador visual complexo no primeiro ciclo e mantém a execução final dentro do ecossistema C#.

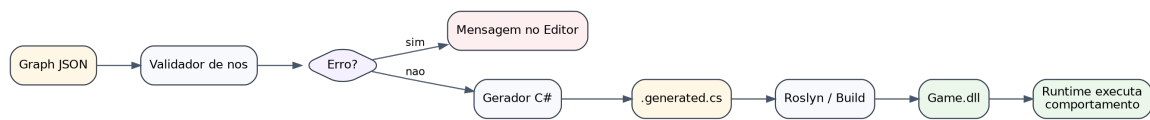


Figura 8: Fluxo de compilação de visual script.

14.1 Modelo do grafo

```

{
  "name": "DoorInteraction",
  "event": "OnInteract",
  "nodes": [
    { "id": "1", "type": "Event.OnInteract" },
    { "id": "2", "type": "Condition.HasItem", "itemId": "OldKey" },
    { "id": "3", "type": "Action.OpenDoor" },
    { "id": "4", "type": "Action.ShowMessage", "text": "Esta trancada." }
  ],
  "connections": [
    { "from": "1", "to": "2" },
    { "from": "2.true", "to": "3" },
    { "from": "2.false", "to": "4" }
  ]
}

```

Listing 7: Exemplo de grafo visual salvo em JSON.

14.2 Código gerado esperado

```
public sealed class DoorInteraction : VisualScriptBehavior
{
    public override void OnInteract(Entity self, Entity interactor, World world
    )
    {
        if (interactor.Inventory.HasItem("OldKey"))
        {
            self.Get<DoorComponent>().Open();
            world.Log.Show("A porta abriu.");
        }
        else
        {
            world.Log.Show("Esta trancada.");
        }
    }
}
```

Listing 8: Exemplo simplificado de C# gerado por visual script.

15 Dados do Jogo

15.1 Definição de entidade

```
{
  "id": "goblin",
  "name": "Goblin",
  "glyph": 34,
  "blocksMovement": true,
  "components": {
    "health": { "max": 10 },
    "combat": { "attack": 3, "defense": 1 },
    "ai": { "behavior": "GoblinAI" }
  }
}
```

Listing 9: Exemplo de entidade em JSON.

15.2 Save game

O save deve registrar estado mínimo para reconstituir o mundo.

```
{
  "projectId": "meu-roguelike",
  "projectVersion": "0.1.0",
  "seed": 982371,
  "floor": 3,
  "player": {
    "entityId": "player",
```

```
"x": 10,  
"y": 22,  
"hp": 30  
},  
"entities": [  
  { "type": "goblin", "x": 12, "y": 20, "hp": 8 }  
]  
}
```

Listing 10: Exemplo de save game.

16 Regras de Negócio

ID	Regra
RN-01	Uma entidade só pode ocupar tile bloqueante se a regra de movimento permitir.
RN-02	Um turno do jogador só deve ser consumido por ações válidas ou explicitamente marcadas como consumidoras de turno.
RN-03	Entidades mortas devem ser removidas ou convertidas em cadáver/item conforme definição do conteúdo.
RN-04	O campo de visão deve diferenciar tile visível, tile explorado e tile desconhecido.
RN-05	A geração procedural deve aceitar seed para depuração e reprodução de bug.
RN-06	Scripts C# não devem manipular diretamente estruturas internas não expostas por API pública da engine.
RN-07	Visual scripts inválidos não devem gerar build; o editor deve indicar o nó problemático.
RN-08	Exportações devem gerar artefatos em pasta separada e nunca sobrescrever o projeto fonte sem confirmação.

17 Critérios de Aceitação do MVP

O MVP será considerado funcional quando cumprir os seguintes itens:

CA-1: Criar um projeto novo pelo template.

CA-2: Abrir uma janela SadConsole com mapa renderizado.

CA-3: Mover o jogador em grid com colisão.

CA-4: Gerar uma dungeon simples com salas e corredores.

CA-5: Criar ao menos um inimigo com IA simples perseguindo o jogador.

CA-6: Executar combate básico entre jogador e inimigo.

CA-7: Carregar entidades a partir de JSON.

CA-8: Compilar pelo menos um script C# do projeto.

CA-9: Gerar pelo menos um visual script simples convertido para C#.

CA-10: Salvar e carregar estado básico do jogo.

CA-11: Exportar uma versão Windows x64 portable executável.

CA-12: Registrar logs de erro em caso de falha de build.

18 Plano de Implementação

18.1 Fase 1 - Núcleo jogável

- Criar solução C#.
- Implementar World, TileMap, Entity e componentes básicos.
- Criar renderer SadConsole mínimo.
- Implementar input e comandos.
- Implementar movimento e colisão.

18.2 Fase 2 - Roguelike mínimo

- Geração procedural simples.
- Sistema de turnos.
- IA básica.
- Combate básico.
- Log de mensagens e UI mínima.

18.3 Fase 3 - Projeto e dados

- Definir `game.mrgproj`.
- Carregar entidades/itens por JSON.
- Criar schema inicial de dados.
- Implementar save/load.

18.4 Fase 4 - Scripting

- Definir interfaces para behaviors.
- Compilar scripts C# no build.
- Mostrar erros amigáveis.
- Vincular scripts a entidades por JSON.

18.5 Fase 5 - Build tool

- Criar comando `momorogue build`.
- Gerar pasta portable.
- Gerar ZIP portable.
- Preparar caminho para single-file.

18.6 Fase 6 - Editor

- Criar editor Avalonia simples.
- Criar/abrir projeto.
- Editar arquivos principais.
- Rodar playtest.
- Acionar build tool.

18.7 Fase 7 - Visual scripting

- Criar editor de grafos mínimo.
- Definir nós: evento, condição, ação, log, spawn, abrir porta.
- Gerar C# a partir do grafo.
- Compilar código gerado no build.

19 Riscos e Mitigações

Risco	Impacto	Mitigação
Escopo crescer demais	Projeto não termina	Manter foco em roguelike e MVP pequeno.
Acoplamento com Sad-Console	Dificulta testes e troca de renderer	Criar interface <code>IRenderer</code> e adaptador separado.
Visual scripting complexo	Atraso grande	Começar com grafo que gera C#, não interpretador completo.
Scripting inseguro	Risco com mods externos	No MVP, aceitar apenas código confiável; sandbox em fase futura.
Build single-file com assets	Assets podem não ser encontrados	Começar com pasta portable; single-file depois.
Editor virar projeto gigante	Perda de foco	Primeiro editor apenas cria, abre, valida, testa e exporta.

Mudança de schema JSON	Projetos antigos quebram	Versionar schemas e criar migrações futuras.
------------------------	--------------------------	--

20 Estratégia de Testes

20.1 Testes unitários

- Movimento e colisão.
- Cálculo de turno.
- Combate.
- FOV.
- Pathfinding.
- Geração procedural com seed fixa.
- Serialização e desserialização.

20.2 Testes de integração

- Carregar projeto completo.
- Compilar scripts C#.
- Gerar visual script e compilar.
- Exportar projeto de exemplo.
- Executar runtime com assets e dados empacotados.

20.3 Testes manuais

- Playtest do template básico.
- Teste de exportação em máquina limpa.
- Validação de UI do editor.
- Teste de mensagens de erro de build.

21 Definição de Pronto

Uma funcionalidade é considerada pronta quando:

1. Possui implementação no módulo correto.
2. Possui teste unitário ou justificativa técnica para teste manual.
3. Não introduz dependência indevida entre engine e UI.

4. Possui logs de erro quando aplicável.
5. Está documentada no README técnico ou em comentário de API pública.
6. Foi validada em projeto de exemplo.

22 Roadmap

Versão	Entrega	Resultado esperado
0.1	Runtime mínimo	Janela, mapa, player, movimento e colisão.
0.2	Roguelike mínimo	Dungeon, turnos, inimigo, combate e log.
0.3	Dados e projeto	<code>game.mrgproj</code> , JSON de entidades/itens e save/load.
0.4	Scripting C#	Scripts compilados e behaviors customizados.
0.5	Build portable	Exportação Windows x64 portable.
0.6	Editor básico	Criar, abrir, editar dados, playtest e exportar.
0.7	Visual scripting MVP	Grafos simples convertidos para C#.
0.8	Instalador	MSI/EXE via WiX.
1.0	Release inicial	Engine utilizável para criar ao menos dois jogos exemplo.

23 Referências Técnicas

- SadConsole. *Welcome to SadConsole*. Disponível em: <https://sadconsole.com/>
- SadConsole CLI templates. Disponível em: <https://sadconsole.com/getting-started/cli/>
- Microsoft Learn. *dotnet publish command*. Disponível em: <https://learn.microsoft.com/en-us/dotnet/core/tools/dotnet-publish>
- Microsoft Learn. *Create a single file for application deployment*. Disponível em: <https://learn.microsoft.com/en-us/dotnet/core/deploying/single-file/overview>
- Avalonia UI Documentation. *Welcome*. Disponível em: <https://docs.avaloniaui.net/docs/welcome>
- Roslyn Scripting API Samples. Disponível em: <https://github.com/dotnet/roslyn/blob/main/docs/wiki/Scripting-API-Samples.md>
- WiX Toolset / FireGiant. Disponível em: <https://www.firegiant.com/wixtoolset/>
- Mermaid CLI. Disponível em: <https://github.com/mermaid-js/mermaid-cli>

A Artefatos de Diagrama

Os diagramas deste documento foram planejados em Mermaid, salvos como arquivos `.mmd` na pasta `diagrams/` e incluídos no PDF como imagens renderizadas na pasta `rendered/`. A entrega acompanha os fontes Mermaid para edição futura.

Arquivo Mermaid	Conteúdo
01_contexto_arquitetura.mmd	Contexto geral da arquitetura.
02_modulos_engine.mmd	Módulos internos da engine.
03_pipeline_build.mmd	Pipeline de build/exportação.
04_sequence_turno.mmd	Sequência de processamento de turno.
05_class_core.mmd	Classes conceituais do núcleo.
06_visual_script.mmd	Fluxo de visual script para C#.
07_estado_runtime.mmd	Estados do runtime.
08_casos_de_uso.mmd	Casos de uso principais.

B Backlog Técnico Inicial

ID	Item	Prioridade
BT-01	Criar repositório e solução .NET	Alta
BT-02	Implementar TileMap e Position	Alta
BT-03	Implementar Entity e Component	Alta
BT-04	Criar renderer SadConsole mínimo	Alta
BT-05	Implementar input para comandos	Alta
BT-06	Implementar geração procedural básica	Alta
BT-07	Implementar turn manager	Alta
BT-08	Implementar IA simples	Média
BT-09	Implementar combate básico	Média
BT-10	Implementar schemas JSON	Média
BT-11	Criar script compiler	Média
BT-12	Criar build tool	Alta
BT-13	Criar editor Avalonia mínimo	Média
BT-14	Criar visual scripting MVP	Baixa/Média
BT-15	Criar instalador WiX	Baixa